

# Pourquoi illinois\_C est lent à grand t — maths de la v7

**Fichier** : maths\_v7\_ntermes\_adaptatif.md · **Dossier** : wiki racine  
**Branche** : master (wiki) · **Auteur** : hprzeta · **MAJ** : 2026-06-11

## 1. La formule de Riemann-Siegel

$$Z(t) = 2 \sum_{n=1}^{N(t)} \frac{\cos(\theta(t) - t \ln n)}{\sqrt{n}} + R(t)$$
$$N(t) = \left\lfloor \sqrt{\frac{t}{2\pi}} \right\rfloor$$

Le nombre de termes  $N(t)$  croît comme  $\sqrt{t}$  : à  $t = 100\,000$ , il faut sommer  $N = \lfloor \sqrt{100000/(2\pi)} \rfloor = 126$  termes.

## 2. Coût de calcul mesuré

Chaque appel à `z_rs_mpr(t)` (35 dps, PREC=116 bits) coûte :

$$t_{\text{appel}}(t) \approx 1.1 \times N(t) \text{ ms} = 1.1 \sqrt{\frac{t}{2\pi}} \text{ ms}$$

| $t$     | $N(t)$ | ms/appel | z/s      |
|---------|--------|----------|----------|
| 1 000   | 13     | ~15 ms   | ~17 z/s  |
| 10 000  | 40     | ~45 ms   | ~8 z/s   |
| 50 000  | 89     | ~98 ms   | ~5 z/s   |
| 77 000  | 111    | ~123 ms  | ~4.3 z/s |
| 100 000 | 126    | ~139 ms  | ~3.8 z/s |

La vitesse chute comme  $1/\sqrt{t}$  : l'algorithme v6 ne peut pas atteindre  $T = 1\,000\,000$  en temps raisonnable sans modification.

### 3. Pourquoi autant d'itérations dans Illinois

Illinois est une méthode de bracketing : à chaque itération, elle évalue  $Z$  en un point  $c$  et réduit l'intervalle  $[a, b]$  contenant le zéro. Environ **10 itérations** suffisent pour converger à  $10^{-12}$ .

**Coût par zéro à  $t = 77\,000$  (v6) :**

$$t_{\text{zéro}}^{v6} = 10 \times 123 \text{ ms} = 1230 \text{ ms} \approx 1.2 \text{ s/zéro}$$

Avec 4 workers, cela donne  $\sim 4.3$  z/s — et le bottleneck mesuré est **illinois\_C = 83 %** du temps CPU total.

### 4. L'erreur de troncature RS

En tronquant à  $M < N(t)$  termes, l'erreur est :

$$|R_M(t)| \lesssim \frac{C}{\sqrt{t}} \cdot \left(\frac{2\pi}{t}\right)^{M/2}$$

Pour détecter le **signe** de  $Z(t)$  (phase de bracketing), une précision  $\epsilon_{\text{sign}} \sim 10^{-3}$  suffit. Cela permet de réduire massivement le nombre de termes pour les premières itérations :

$$N_{\text{fast}}(t) = \max\left(5, \left\lfloor \frac{N(t)}{4} \right\rfloor\right)$$

À  $t = 77\,000$  :  $N_{\text{fast}} \approx 28$  au lieu de 111. **Gain  $\times 4$  sur les 8 premières itérations.**

### 5. Architecture v7 — Illinois adaptatif 2 phases

**Phase 1 — Bracketing rapide (itérations 1 à 8)**

$$N = N_{\text{fast}} \approx \frac{\sqrt{t/2\pi}}{4}, \quad \text{prec} = 64 \text{ bits}, \quad \text{tol} = 10^{-4}$$

Coût :  $28 \times 0.08 \text{ ms} \approx 2.2 \text{ ms/appel}$

Cette phase réduit  $[a, b]$  jusqu'à une largeur  $< 10^{-4}$  avec très peu de calcul. Le signe de  $Z$  peut être évalué avec seulement  $N/4$  termes car les termes négligés sont d'ordre  $\left(\frac{2\pi}{t}\right)^{N/8} \ll 1$ .

**Phase 2 — Polish précis (itérations 9 à convergence)**

$$N = N(t) = \sqrt{\frac{t}{2\pi}}, \quad \text{prec} = 116 \text{ bits (35 dps)}, \quad \text{tol} = 10^{-12}$$

Coût :  $111 \times 1.1 \text{ ms} \approx 123 \text{ ms/appel}$

Seulement **2 itérations** de phase 2 suffisent pour atteindre  $10^{-12}$  (l'intervalle est déjà resserré à  $10^{-4}$  par la phase 1).

## 6. Gain théorique v7

$$t_{\text{zéro}}^{v7} = 8 \times 2.2 \text{ ms} + 2 \times 123 \text{ ms} = 17.6 + 246 = 264 \text{ ms}$$

$$\text{Gain} = \frac{t_{\text{zéro}}^{v6}}{t_{\text{zéro}}^{v7}} = \frac{1230}{264} \approx \times 4.6$$

| Métrique                | v6       | v7 estimé      |
|-------------------------|----------|----------------|
| ms/zéro à $t = 77\,000$ | 1230 ms  | ~264 ms        |
| z/s à $t = 77\,000$     | 4.3 z/s  | ~20 z/s        |
| Durée $T = 100\,000$    | ~130 min | <b>~28 min</b> |

## 7. Implémentation — `illinois_refine_adaptive()`

La fonction C ajoutée dans `illinois_mpfr.c` :

```
double illinois_refine_adaptive(
    double a, double b,
    double fa, double fb,
    double t,
    int iter_switch, /* numéro d'itération bascule phase 2 */
    int max_iter
)
```

- **iter\_switch** : calibré empiriquement via benchmark  $T=5\,000$  (théorique : 8, à mesurer pour confirmer)
- Bascule de précision via `mpfr_prec_round()` à l'itération `iter_switch`
- Fonction auxiliaire `Z_rs_mpfr_ntermes(t, N_termes, prec)` acceptant  $N$  variable

## 8. Ce qui reste incertain

- **iter\_switch optimal** : 8 est théorique → à calibrer par benchmark  $T=5\,000$
- **Erreur signe phase 1** : si  $N_{\text{fast}}$  trop petit à très grand  $t$ , risque de fausse convergence (détection de signe incorrecte)
- **Gain réel vs théorique** : la phase 2 peut nécessiter plus de 2 itérations si la phase 1 n'a pas convergé assez (dépend de la courbure de  $Z$  au voisinage du zéro)

## 9. Questions ouvertes (v8+)

- $T = 1\,000\,000$  :  $N(t) \approx 400 \rightarrow$  Illinois encore viable ?
- Précision 64 bits suffit-elle pour le signe à tout  $t$  ?
- Gain combiné  $W = 8 (\times 1.3) + \text{illinois adaptatif} (\times 4.6) = \times 6 \rightarrow \sim 20 \text{ min } T = 100\,000$  ?

- Remplacement par méthode de Newton (convergence quadratique) pour  $t > 500\,000$  ?

---

*maths\_v7\_ndermes\_adaptatif.md · wiki racine · master · hprzeta · MAJ 2026-06-11*  
· ~120 lignes